NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AI Integration Report

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 10/08/2026

Team Information

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Team Leader	M.M. Asfy Or Rayhan
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Contents

1. **Selected AI Platform** ……..……...……..1

2. **AI Model Used**……...……..…..6

3. **AI Feature Implemented** ……...……..…….....10

4. **AI Workflow**……..……..……...……...…16

5. **Prompt Engineering Examples**……....……...……..20

6. **AI Response Handling Strategy**……....……...……..26

7. **Current Development Progress**……....……...……..30

8. **GitHub Repository Link** ……...……..…...……..33

AI Integration Report

Project Title: AI Programming Tutor & Bug Explainer

1. Selected AI Platform

1.1 Name of the AI Platform

The AI platform used by the AI Chat feature is Hugging Face, accessed through its Inference Providers router. This is decided by a constant named `FEATURE_PROVIDER` inside `backend/server/src/services/aiService.js`:

```
const FEATURE_PROVIDER = {
  // Hugging Face — the rest
  tutor:            'huggingface',      <-- the AI Chat feature
  bugExplainer:     'huggingface',
  errorTranslator:  'huggingface',
  codeImprover:     'huggingface',
  problemGenerator: 'huggingface',
};
```

The code comment in the same file states the reason for this choice: Hugging Face is used because it provides a free Inference API.

1.2 How the Platform Is Integrated Into the Project

The integration is not written directly inside the chat controller. The project uses a provider abstraction layer, which means all AI platform code lives in one file and the rest of the application does not know or care which platform is used. The chat controller only calls one function:

```
const response = await askAI('tutor', { message, memory });
```

Inside `askAI()`, the helper `providerFor(feature)` looks up the feature in `FEATURE_PROVIDER` and returns the platform name. The advantage of this design is that the AI Chat can be moved to a different platform by changing one word in the map, with no change to the controller, the route, or the frontend. The project also includes complete working code for four other platforms — Anthropic (Claude), Google Gemini, OpenAI, and Groq — plus a built-in offline Mock provider. These are available but are not used by the AI Chat feature by default. An environment variable named AI_PROVIDER can override the map for every feature at once.

1.3 How the Backend Communicates With the AI Service

Hugging Face is grouped in the code under a configuration object named OPENAI_COMPAT, because Hugging Face's router speaks the same /chat/completions format as OpenAI and Groq. The configuration is:

```
huggingface: {

  // Hugging Face Inference Providers — OpenAI-compatible
router endpoint.

  baseURL: 'https://router.huggingface.co/v1',

  key:     () => process.env.HF_API_KEY ||
process.env.HUGGINGFACE_API_KEY,

  model:   () => process.env.HF_MODEL || 'meta-llama/Llama-
3.1-8B-Instruct',

},
```

The following configuration shows how the backend connects to Hugging Face and which settings are used for the AI Chat feature. It defines the API endpoint, API key, and default AI model used to process user questions.

The actual network request is made by the function callOpenAICompat(). It is a standard HTTPS POST request built with the built-in fetch API:

Property	Value Used in the Code
Method	POST
URL	https://router.huggingface.co/v1/chat/completions
Header: Content-Type	application/json
Header: Authorization	Bearer <HF_API_KEY from the .env file>
Body: model	meta-llama/Llama-3.1-8B-Instruct
Body: messages	An array of two objects — one with role "system" and one with role "user"
Body: temperature	0.4
Timeout wrapper	fetchWithTimeout() with AbortController, default 30,000 ms

No official SDK or npm package is used for the AI call. The backend talks to Hugging Face directly over HTTP using fetch. The package.json for the backend confirms this — it lists express, mongoose, jsonwebtoken, bcryptjs, multer, unpdf, cors, dotenv, and express-rate-limit, but no AI client library.

1.4 Where the AI Integration Is Implemented

The AI Chat feature is implemented across several backend and frontend files, with each file handling a specific part of the overall process. This separation keeps the system organized and makes the AI integration easier to maintain and modify. The table below shows the main files involved in the AI Chat integration and explains the role of each file in the system.

File	Role in the AI Chat Integration
backend/server/src/services/aiService.js	The AI integration layer. Holds the platform map, the API keys, the endpoint URLs, the timeout wrapper, the network call, and the fallback logic.
backend/server/src/services/prompts.js	Holds the system prompt for the chat and builds the user message.
backend/server/src/services/memoryService.js	Loads the learner's past context that is added into the chat prompt.
backend/server/src/controllers/aiController.js	The tutor() function — validates the input, calls askAI(), saves history, awards XP.
backend/server/src/routes/aiRoutes.js	Registers POST /api/ai/tutor with the optionalProtect middleware.
backend/server/src/server.js	Mounts the route and applies the rate limiter.
frontend/client/src/pages/Tutor.jsx	The chat user interface.
frontend/client/src/api/client.js	The Axios call that sends the question to the backend.
frontend/client/src/components/AIResultCard.jsx	Displays the AI answer.
frontend/client/src/components/Markdown.jsx	Renders bold text and code blocks in the answer.

1.5 A Documentation Mismatch That Should Be Corrected

The project Readme.md contains an “AI Providers” table that lists Groq, Google Gemini, OpenAI, and Mock, and its setup instructions tell the user to set AI_PROVIDER=groq. The Readme does not mention Hugging Face at all. The source code, however, assigns Hugging Face to the chat feature. This report follows the source code, because the code is the real implementation. The Readme should be updated so the documentation matches the code.

2. AI Model Used

2.1 Model Name

The AI model used by the AI Chat feature is `meta-llama/Llama-3.1-8B-Instruct`. This is the default model name written directly in the source code.

2.2 Where the Model Is Configured

The model name is set in the `OPENAI_COMPAT` configuration object in `backend/server/src/services/aiService.js`, on this line:

```
model: () => process.env.HF_MODEL || 'meta-llama/Llama-3.1-8B-Instruct',
```

The model is configured in two layers, which is good practice:

- **Environment variable (first choice):** `HF_MODEL` read from the `.env` file. This allows the model to be changed without editing any code.
- **Hard-coded default (fallback):** `meta-llama/Llama-3.1-8B-Instruct`, used whenever `HF_MODEL` is not set.

Limitation to note: the archive does not contain a backend .env or .env.example file, so the variable HF_MODEL is not documented anywhere in the project itself. In practice this means the hard-coded default is the model that will run.

2.3 How the Model Is Called by the Backend

The model name is placed in the JSON body of the request to Hugging Face. This is the actual code that performs the call:

```
async function callOpenAICompat(provider, systemPrompt, userMessage) {
  const cfg = OPENAI_COMPAT[provider];
```

```
  const res = await
fetchWithTimeout(`${cfg.baseURL}/chat/completions`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json',
               Authorization: `Bearer ${cfg.key()}` },
    body: JSON.stringify({
      model: cfg.model(),                              // the
model name goes here
      messages: [
        { role: 'system', content: systemPrompt },      // the
chat personality
        { role: 'user',   content: userMessage },       // the
student's question
      ],
      temperature: 0.4,
    }),
  });


  if (res.status === 429) throw new Error(`${provider} rate
limit hit (429)`);
  if (!res.ok) throw new Error(`${provider} API error:
${res.status} ${await res.text()}`);
  const data = await res.json();
  return data?.choices?.[0]?.message?.content ?? '';   // only
the text is kept
}
```

Two settings are worth explaining during a viva:

- **temperature: 0.4** — a low value. Temperature controls how random the model's wording is. A low value was chosen because the chat answer must follow a fixed section structure that the frontend renders, so consistency matters more than creativity.

- **Optional chaining (?.) in the response reading** — if Hugging Face returns a body that does not contain choices[0].message.content, the code produces an empty string instead of crashing the server.

Overall, this function provides a simple and reliable way for the backend to communicate with the selected AI model. It sends the system instructions and the student's question to Hugging Face, checks for common API errors such as rate limiting, and extracts only the generated text from the response. As a result, the controller receives a clean AI-generated answer that can be saved to the chat history and displayed to the student through the frontend.

2.4 What Role the Model Performs in the AI Chat

The model is the main component of the AI Chat feature that understands the student's question and generates the teaching-oriented response. In this project, it can read free-form programming questions written in English, Bangla, or Banglish and understand the requested programming concept even though the system does not use a predefined list of topics. Based on the student's question, the model generates a one-line simple definition of the concept, provides a suitable everyday real-life analogy, and produces a small and correct code example inside a fenced code block. It also adjusts the depth and difficulty of the explanation based on the learner's private context that is added to the prompt. Furthermore, the model automatically chooses the response language by mirroring the language used by the student. Since the selected model is an instruction-tuned chat model, it is well suited to the project because the system prompt provides specific instructions and formatting rules that the model is expected to follow when generating each response. This role makes the model more than a simple question-answering tool, because it provides personalized explanations that are focused on helping students understand programming concepts. It also allows the AI Chat feature to act like a virtual programming tutor that can respond interactively to different learning needs.

2.5 Other Models Present in the Code but Not Used by the Chat

For completeness, aiService.js contains default model names for the other supported platforms. None of these are used by the AI Chat feature under the default configuration.

Platform	Default Model in Code	Used by AI Chat?
Hugging Face	meta-llama/Llama-3.1-8B-Instruct	Yes — this is the chat model
Hugging Face (vision)	meta-llama/Llama-3.2-11B-Vision-Instruct	No — images are not part of the chat
Anthropic	claude-3-5-haiku-latest	No — used for repo and document features
Google Gemini	gemini-1.5-flash	No
OpenAI	gpt-4o-mini	No
Groq	llama-3.3-70b-versatile	No

3. AI Feature Implemented

3.1 What the AI Chat Feature Does

The AI Chat is a question box where a student can type any programming concept question in plain language and receive a beginner-friendly written explanation. The page header in Tutor.jsx describes it as, “Ask any programming concept. Answers stay simple and beginner-friendly.” The interface is divided into four main parts. First, there are Quick Example Buttons that provide four ready-made questions: “What is an array?”, “Explain recursion simply”, “What is OOP?”, and “Binary search example”, allowing students to ask a question without typing it manually. Second, there is a Text Input Box with the placeholder “e.g. What is a linked list?”. The student can press the Enter key to submit the question, making the interface feel similar to a chat box. Third, there is an Ask Button, which becomes disabled while a request is in progress to prevent the student from accidentally sending the same question multiple times. Finally, there is an Answer Card, implemented through AIResultCard, which displays an animated “Thinking like a tutor…” indicator while the AI is processing the request and then shows the formatted answer. The answer card also includes a Listen button that allows the student to hear the AI-generated explanation aloud.

3.2 What Problem It Solves

The project Readme lists the first objective as: “Lower the barrier to learning programming — provide instant, plain-language explanations of programming concepts so that absolute beginners never feel lost or discouraged.” The AI Chat is the feature that delivers this objective. The real problem is that a beginner who does not understand a concept usually has only three options: wait for the next class, search the internet and land on material written for experienced developers, or give up. Textbooks and online tutorials are written once, for a general reader, and mostly in English. A student in Bangladesh who is confused about pointers at 11 p.m. has no one to ask. The AI Chat gives that student an answer immediately, at a beginner reading level, and in the language they are comfortable with — Bangla, Banglish, or English. The system prompt instructs the model to be “warm and encouraging” and to “never make the learner feel judged or slow”, which

addresses the emotional side of the problem: many beginners stop asking questions because they are embarrassed.

3.3 Why AI Is Required for This Feature

This feature cannot be effectively built without AI, which is an important point to explain and defend during a viva. The input is unlimited because a student can ask about any programming concept in at least three languages and can phrase the question in many different ways. Since there is no fixed list of possible questions, a database containing pre-written answers or a rule-based system based on keyword matching would fail when an unexpected question is asked. The output also needs to be generated rather than simply retrieved, because the response must include a suitable analogy and a code example specifically created for the concept asked by the student, which cannot be reliably stored in a predefined table. Furthermore, the answer needs to be rewritten according to the learner's individual needs. For example, the same concept may need to be explained differently for a student whose recorded weak topics include recursion compared with a student who has no difficulty with recursion. A language model can dynamically adjust the explanation and difficulty based on the available learner context. Finally, the reply language must be selected automatically. The prompt instructs the model to "mirror the learner," meaning it needs to understand whether the student's question is written in English, Bangla, or Banglish and generate the response in a suitable matching language.

3.4 What Users Can Ask the AI

There is no restriction in the code on what a user may type. The only check performed is that the message is not empty. The system prompt frames the task as "Answer the learner's programming-concept question clearly", so the AI is directed towards programming concepts, and the example buttons show students the kind of question that is expected:

Example Question (from the project code)	Where It Appears
What is an array?	Example button in Tutor.jsx

Example Question (from the project code)	Where It Appears
Explain recursion simply	Example button in Tutor.jsx
What is OOP?	Example button in Tutor.jsx
Binary search example	Example button in Tutor.jsx
e.g. What is a linked list?	Input box placeholder in Tutor.jsx

Note: there is no topic filter, no moderation, and no check that the question is actually about programming. If a student asks something unrelated, only the wording of the system prompt discourages an off-topic answer. Input topic validation is: Not implemented.

3.5 What Type of Programming Assistance the AI Provides

The chat provides conceptual teaching assistance. The system prompt forces every answer into the same four-part shape, so the student always receives the same kind of help:

Section of the Answer	What the Student Gets
Simple definition	A one-line plain-language meaning of the concept, with no jargon.
Analogy	One short everyday real-world comparison that makes the idea click.
Example	A small, clean, correct code snippet inside a fenced code block.
Keep going	One encouraging sentence, to keep the beginner motivated.

It is worth stating what the chat does not do, because the project has separate features for those tasks. The chat does not debug pasted code (that is the Bug Explainer), does not translate compiler

errors (the Error Translator), does not review or refactor working code (the Code Improver), does not read files or images (the Document and Repo Explainers). The chat is deliberately limited to concept explanation.

3.6 How the Chat Is Personalised (and Its Limits)

The chat is connected to the project's memory system, which is what separates it from a plain chatbot. Two things happen around every chat message, in aiController.js:

```
export async function tutor(req, res) {
  const { message } = req.body;
  if (!message) return res.status(400).json({ message:
'message is required' });


  const memory   = await getContext(req.user?._id);
// READ past context
  const response = await askAI('tutor', { message, memory });
  await saveHistory({ userId: req.user?._id, feature:
'tutor',
                      message, response, topic: message });
  await addNote(req.user?._id,
// WRITE new context
                `Asked the tutor about: "${message.slice(0,
80)}"`);
  await awardXP(req.user, 5);
  res.json({ response });
}
```

- **getContext()** reads the student's last 6 memory notes and their top 5 weak topics from MongoDB and builds a short context paragraph, which is added to the prompt.
- **addNote()** saves a note such as "Asked the tutor about: Explain recursion simply". Only the 40 most recent notes are kept, using a MongoDB $push with $slice: -40.

Three Honest Limits of This Personalisation

There are three important limitations of the current personalisation system that should be honestly explained during the viva. First, personalisation works only for a logged-in student and when the database is connected. For a guest user, `getContext()` returns an empty string and `addNote()` does nothing, so the guest receives a normal, non-personalised answer. Second, the current implementation does not write any data to the `WeakTopic` collection. Although `getContext()` can read information from this collection, no controller currently creates or increments a `WeakTopic` record. Therefore, in practice, the "topics they often struggle with" part of the learner context remains empty. Third, the memory notes contain only short summaries of what the student asked. The actual previous questions and AI-generated answers are stored in `ChatHistory`, but they are not sent back to the model as context. Because of this limitation, the current AI Chat does not properly support follow-up questions that depend on the previous conversation.

3.7 How the Chat Supports the Project Objectives

Project Objective (from the Readme)	How the AI Chat Supports It
1. Lower the barrier to learning programming	Directly delivered — this is the feature that gives instant plain-language explanations of concepts.
2. Personalise the learning experience	Supported — the chat both reads the learner memory before answering and writes a new note after answering.
3. Gamify and motivate learning	Supported — each chat message awards 5 XP, and the level is recalculated as Math.floor(xp / 100) + 1.
4. Provide accessible AI education	Supported — the chat runs on a free-tier platform, works without login, and falls back to an offline Mock answer when no API key is set.

Project Objective (from the Readme)	How the AI Chat Supports It
5. Empower administrators	Supported indirectly — every chat message is saved to ChatHistory, which feeds the admin activity list and the usage statistics chart.

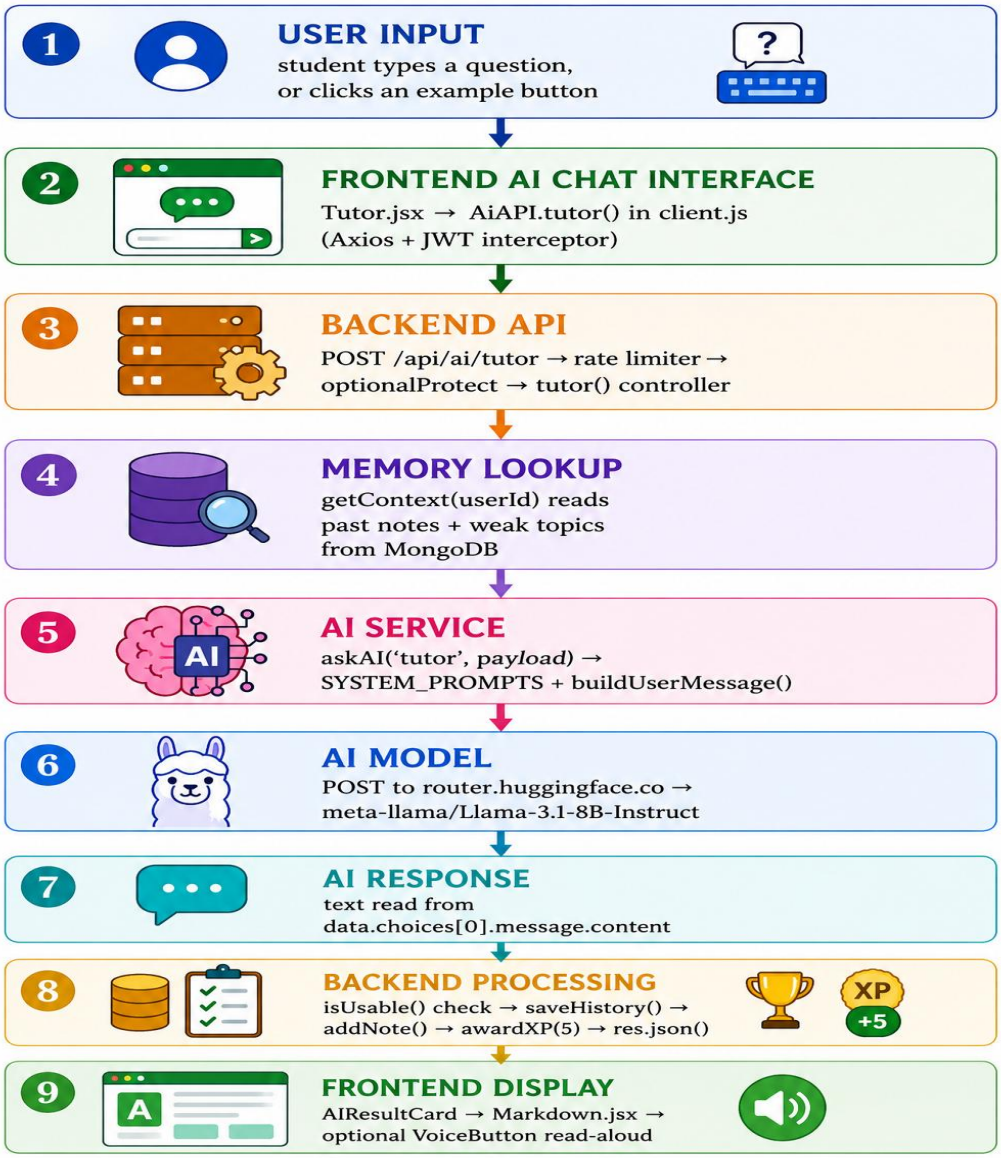
Overall, the AI Chat is closely aligned with the project objectives because it provides instant programming assistance, introduces basic learner personalisation, encourages engagement through XP, supports accessible AI-based learning, and generates useful activity data for administrators. However, the personalisation objective is only partially achieved in the current implementation because some planned memory and weak-topic features are not yet fully connected to the AI response process.

3.8 How the Feature Adds Meaningful Value

The AI Chat is not included only to satisfy a course requirement; several design decisions in the implementation show that it was developed to be genuinely useful for students. First, the feature works without requiring an account. The route uses optionalProtect instead of protect, allowing a student who is stuck on a programming problem to get immediate help without registering. Login is only required for additional features such as chat history, XP, and personalisation. Second, the AI Chat supports Bangla and Banglish in addition to English. The base prompt instructs the model to mirror the language used by the student, which helps reduce the language barrier for local students. Finally, the feature includes voice output through VoiceButton.jsx, which uses the browser's built-in SpeechSynthesis API to read the AI-generated answer aloud. Before speaking, the system removes unnecessary Markdown symbols and replaces code blocks with the phrase "(code example shown on screen)" so that the spoken response sounds more natural. Since this feature uses the browser's built-in functionality and does not require an additional paid service, it provides a cost-free accessibility option for students who may find reading English explanations more difficult than listening to them.

4. AI Workflow

The diagram below shows the exact path a chat message takes in this project, with the real file and function names from the code.



Step 1 — User Input

The student either types a question into the input box and presses Enter or the Ask button, or clicks one of the four example buttons. Clicking an example both fills the input box and sends the question in one action.

Step 2 — Frontend AI Chat Interface

The ask() function in Tutor.jsx runs. It first checks that the message is not empty using if (!message.trim()) return;, then sets loading to true and clears the previous answer so the old response does not stay on screen. It then calls AiAPI.tutor(message). In client.js this becomes an Axios POST to /ai/tutor with the body { message }. A request interceptor automatically reads the JWT token from localStorage under the key cm_token and, if it exists, attaches it as an Authorization: Bearer header. This is how the backend knows whether the person is a logged-in student or a guest.

Step 3 — Backend API

The Express server receives the AI Chat request, and two important processes take place before the request reaches the controller. First, the rate limiter provided by express-rate-limit is applied to /api/ai and allows a maximum of 30 requests per minute, with standard rate-limit headers enabled. This helps protect the project's free AI API quota from being consumed too quickly and prevents excessive requests to the AI service. Second, the optionalProtect middleware checks whether a valid JWT is available. If a token is present and valid, it attaches the authenticated user information to req.user; however, the middleware uses a try/catch mechanism that ignores authentication failures and does not block the request. This behavior is important because it allows the AI Chat to work for both logged-in students and guest users. After these middleware processes are completed, the tutor() controller validates the request input. If the required message field is missing, the controller returns an HTTP 400 response with the message **"**message is required".

Step 4 — Memory Lookup

getContext(req.user?._id) is called. For a guest, or when the database is not connected, it returns an empty string immediately. For a logged-in student it reads the last 6 memory notes and the top 5 weak topics (sorted by mistakeCount) and builds a short context paragraph. This whole function is wrapped in a try/catch that returns an empty string on any error, so a database problem cannot break the chat.

Step 5 — AI Service and Prompt Assembly

The askAI('tutor', { message, memory }) function performs four main operations in sequence. First, it looks up the system prompt associated with the key “tutor” from SYSTEM_PROMPTS. If the specified key does not exist, the function throws an “Unknown AI feature” error. Second, it uses buildUserMessage() to construct the user message by placing the learner’s memory context first, followed by the line “Question:” and the student’s actual question. Third, the function determines which AI provider should be used by calling providerFor('tutor'), which returns “huggingface” for the AI Chat feature. Finally, it checks whether the Hugging Face provider has a valid API key through providerHasKey('huggingface'). If neither HF_API_KEY nor HUGGINGFACE_API_KEY is configured, the backend does not contact the external AI service. Instead, it returns a built-in Mock answer, allowing the application to continue working even when an API key is unavailable.

Step 6 — AI Model Call

callOpenAICompat() sends the POST request described in Section 2.3 to Hugging Face. The request is wrapped in fetchWithTimeout(), which creates an AbortController and a timer set to AI_TIMEOUT_MS (30,000 ms by default). If the timer fires first, the request is aborted and the error is converted into the readable message “AI request timed out after 30000ms”. The timer is always cleared in a finally block.

Step 7 — AI Response

Hugging Face returns a JSON body. Before reading it, the code checks the status: a 429 becomes a rate-limit error, and any other non-OK status becomes an API error containing the status code and the response body. If the status is fine, only the text is extracted with data?.choices?.[0]?.message?.content ?? ‘’ and everything else in the response is discarded. This ensures that the frontend receives only the actual AI-generated answer rather than the complete API response. The optional chaining also prevents the server from crashing if the expected response structure is missing. The extracted text is then passed back to the controller for validation and further processing.

Step 8 — Backend Processing

The extracted AI response is first passed through isUsable(), which checks whether the response is a valid string with a non-empty trimmed length. If the response fails this check, the system uses a built-in Mock answer instead. After obtaining a usable answer, the controller performs three best-effort tasks, each protected so that a failure in these tasks cannot prevent the AI response from being returned to the student. First, saveHistory() creates a ChatHistory document containing the feature name “tutor”, the student’s question, the generated answer, and the topic. This operation is skipped for guest users and is also protected by try/catch. Second, addNote() saves a short memory note that can be used for future personalisation. Third, awardXP(req.user, 5) adds 5 XP to the logged-in student and recalculates the student’s level. This operation is also wrapped in an empty catch block, meaning that an XP-related failure is silently ignored and does not affect the AI response. Finally, after these processing steps are completed, the controller sends the generated answer to the frontend using res.json({ response }).

Step 9 — Frontend Display

The page stores the text in state and clears the loading flag in a finally block, so the loading indicator always disappears even if the request failed. AIResultCard then renders the answer through Markdown.jsx, which splits the text on triple backticks, renders the odd blocks as styled code, and converts **bold** markers into bold text. A Framer Motion fade-in animation is applied, and a VoiceButton appears so the student can listen to the explanation.

5. Prompt Engineering Examples

All prompts for the whole project are kept in one file, backend/server/src/services/prompts.js. The prompt used by the AI Chat is built in two parts: a shared BASE_RULES block that every feature inherits, and a short TASK block that is specific to the chat.

5.1 System Prompt — Part 1: Shared Base Rules

This is the actual BASE_RULES text from the project, used exactly as written:

```
You are "BugTutor AI", a kind, patient programming teacher
for absolute beginners.


CORE TEACHING RULES (always follow):
- Explain in very simple language, as if teaching a first-
year student.
- Avoid jargon. If a technical term is unavoidable, define it
in one short line.
- Be warm and encouraging. Never make the learner feel judged
or slow.
- Prefer a small real-world analogy when it makes an idea
click.
- Include a short, clean, correct code example whenever it
helps.
- Stay focused and concise. Never dump a wall of text.


LANGUAGE:
- You may use Banglish (Bangla + English) when it aids
understanding.
- Mirror the learner: if they write in Bangla, lean Bangla;
if English, lean English.
- Follow any explicit language instruction given in the
request.
```

```
FORMATTING:
- Use short Markdown sections with bold labels exactly as specified in the TASK.
- Keep code inside fenced code blocks with the correct language tag.


PERSONALIZATION (strict):
- You may receive private context about the learner. Use it silently to tailor
  depth, examples, and difficulty.
- NEVER mention, describe, quote, or hint that you remember the learner or have
  prior context. Do not write phrases like "I remember", "as we discussed",
  "you struggled with", or the Bangla equivalents. Just give a better answer.
```

5.2 System Prompt — Part 2: The Chat TASK Block

This is the actual TASK block for the tutor feature. The final system prompt sent to the model is BASE_RULES followed by this text:

```
TASK: Answer the learner's programming-concept question clearly.

Reply with these bold sections, in order:

**Simple definition** — one line.

**Analogy** — one short everyday comparison.

**Example** — a tiny code snippet in a fenced code block.

**Keep going** — one encouraging sentence.
```

5.3 How the User Prompt Is Built Dynamically

The user prompt is not simply the student's question. It is assembled at runtime by buildUserMessage() in prompts.js. This is the actual code for the tutor case:

```
export function buildUserMessage(feature, payload) {
  const memoryBlock = memoryDirective(payload.memory);
  switch (feature) {
    case 'tutor':
      return `${memoryBlock}Question: ${payload.message}`;
    ...
}


// memoryDirective() — returns an empty string when there is
no memory
function memoryDirective(memory) {
  if (!memory) return '';
  return 'PRIVATE CONTEXT ABOUT THIS LEARNER (for your eyes
only — use it to tailor ' +
         'depth and examples, but NEVER mention, quote, or
hint that you have it):\n' +
         `${memory}\n\n---\n`;
}
```

So the user prompt has two possible shapes depending on who is asking:

- **Guest, or a logged-in student with no saved history:** the memory block is an empty string, and the prompt is only the line `Question: <the student's text>`.
- **Logged-in student with saved history:** the private context block is placed first, followed by a `---` separator, and then the question.

This dynamic approach allows the AI Chat to provide both standard and personalised responses. The learner's question remains the main input, while memory is added only when relevant context is available. This keeps the prompt flexible and maintains a consistent response structure for all users.

5.4 Complete Example — Guest User

User Prompt actually sent to the model:

```
Question: Explain recursion simply
```

Expected AI Output:

A short beginner-level answer containing exactly four bold sections in this order: Simple definition (one line), Analogy (one everyday comparison), Example (a tiny code snippet inside a fenced code block), and Keep going (one encouraging sentence). The language should mirror the question, so an English question should get an English answer. No jargon should be used without a one-line definition, and the answer should be short rather than a wall of text.

5.5 Complete Example — Logged-In Student With Memory

User Prompt actually sent to the model (the context values shown here are examples of what getContext() would produce from the note format used by the code):

```
PRIVATE CONTEXT ABOUT THIS LEARNER (for your eyes only — use
it to tailor
depth and examples, but NEVER mention, quote, or hint that
you have it):
Recently this student worked on:
- Asked the tutor about: "What is an array?"
- Debugged some cpp code


---
Question: Explain recursion simply
```

Expected AI Output:

The same four-section structure as above, but with the explanation quietly adjusted to suit a student who has recently been working with arrays and C++ — for example, by choosing a C++ code example. Critically, the answer must not contain any sentence such as "Since you were working on arrays earlier…", because the prompt strictly forbids the model from revealing that it has any prior context.

5.6 Prompt Engineering Techniques Used

Technique	How It Appears in the Chat Prompt	Why It Helps
Role assignment	"You are BugTutor AI, a kind, patient programming teacher for absolute beginners."	Fixes the tone and the reading level of every answer.
Reusable base template	BASE_RULES is written once and shared by all eight features.	Keeps behaviour consistent and lets the team change the personality in one place.
Explicit output structure	The TASK names the four bold sections and the required order.	Makes the output predictable so Markdown.jsx can render it cleanly.
Negative instruction	"NEVER mention, describe, quote, or hint that you remember the learner"	Stops the personalisation from feeling intrusive or robotic.
Length control	"Stay focused and concise. Never dump a wall of text."	Prevents an overwhelming answer for a beginner.
Adaptive language rule	"Mirror the learner: if they write in Bangla, lean Bangla"	One prompt serves Bangla, Banglish, and English students.

Technique	How It Appears in the Chat Prompt	Why It Helps
Context injection	memoryDirective() wraps learner memory as private context above the question.	Personalises the depth of the answer without extra prompts.
Low temperature	temperature: 0.4 in the API request	Reduces randomness so the required structure is followed more often.

5.7 What Is Not in the Chat Prompt

There are also some limitations in the current prompt design. First, an explicit language directive is available through the languageDirective() helper in prompts.js, which converts a selected language into a firm instruction; however, this helper is currently applied only to the repository and document features. The AI Chat instead relies on the softer "mirror the learner" rule defined in BASE_RULES, and the chat interface does not provide a language selector. Second, the prompt does not use few-shot examples, meaning it contains no sample question-and-answer pairs to demonstrate the expected response format and relies entirely on written instructions. Finally, conversation history is not included in the prompt, so previous chat messages are not sent to the model. As a result, the current implementation does not support context-aware follow-up questions.

6. AI Response Handling Strategy

This section reports only the handling that exists in the code. Where something is missing, it is stated plainly.

6.1 Summary Table

Scenario	Status	How It Is Handled
Successful response	Implemented	The text is read from data.choices[0].message.content, checked by isUsable(), saved to history, and returned as res.json({ response }). The frontend renders it in AIResultCard.
Invalid response	Not implemented	For the chat there is no structure check. The code never verifies that the four required bold sections are present. Any non-empty text is accepted and displayed as it is.
Empty response	Implemented	isUsable(text) requires a string with a non-empty trimmed length. If the model returns an empty or whitespace-only reply, askAI() returns the Mock answer instead of showing a blank card.
API errors	Implemented	if (!res.ok) throws an error containing the status code and body. askAI() catches it, logs it to the server console, and returns the Mock answer. The request never crashes.
Network failures	Implemented	A failed fetch throws and is caught by the same try/catch, again returning the Mock answer. On the frontend an Axios failure is caught and a message is shown to the student.

Scenario	Status	How It Is Handled
Rate limits	Partially implemented	HTTP 429 is explicitly detected and turned into a clear error message, and express-rate-limit protects the project's own endpoint at 30 requests per minute. But there is no retry, no backoff, and no queue — a 429 simply falls back to the Mock answer.
Timeout	Implemented	fetchWithTimeout() uses an AbortController with AI_TIMEOUT_MS (30,000 ms default) and converts an AbortError into "AI request timed out after 30000ms", which is then caught and replaced by the Mock answer.

6.2 The Single Pattern Behind All of It

Almost the entire strategy is contained in one block at the end of askAI(). This is the real code:

```
try {
  if (provider === 'mock' || !providerHasKey(provider))
return callMock(feature, payload);
  const out = await runText(provider, systemPrompt,
userMessage);
  // Empty-response guard — fall back instead of showing a
blank card.
  return isUsable(out) ? out : callMock(feature, payload);
} catch (err) {
  // Never crash the request because of an AI hiccup —
degrade gracefully.
  console.error(`[aiService] ${feature} via ${provider}
failed, using mock:`, err.message);
  return callMock(feature, payload);
}
```

This one pattern covers a missing API key, an empty reply, an API error, a network failure, a rate limit, and a timeout. The design goal, stated in the code comment, is graceful degradation: the AI Chat always returns something usable, so the application keeps working even with no internet and no API key.

6.3 What the Mock Fallback Returns for the Chat

When any failure happens, callMock(‘tutor’, payload) returns a pre-written demo answer that follows the same four-section shape, includes the student’s question text, and ends with the line: “(Demo response — set an API key in .env for real answers.)”

6.4 An Important Weakness to Acknowledge

Graceful degradation is a genuine strength, but it has a side effect that should be admitted honestly in a viva. When a real AI call fails, the backend still replies with HTTP 200 and a Mock answer. The response body contains no flag such as usedFallback, so the frontend cannot tell a real AI answer from a fallback answer, and the student is never clearly told that the AI was unavailable. The only clue is the demo line inside the Mock text itself. All AI errors are logged to the server console only, where a student user will never see them.

6.5 Frontend Error Handling in the Chat Page

Tutor.jsx wraps its request in a try/catch/finally:

```
try {
  setResult(await AiAPI.tutor(message));
} catch {
  setResult('**Something went wrong.** Is the backend
running?');
} finally {
  setLoading(false);
}
```

This catch only fires when the request to the project's own backend fails — for example, if the server is not running or the browser has no connection. It cannot fire because of an AI provider failure, since those are already converted into a successful HTTP 200 Mock response by the backend. The finally block guarantees the loading indicator is removed in every case. Note that the catch block does not read the specific server error message; it shows one fixed sentence for every failure.

6.6 Not Implemented for the AI Chat

The current AI Chat implementation also has several limitations in terms of reliability and response management. It does not provide automatic retry or exponential backoff after a failed or rate-limited API call, and it cannot automatically fail over to a second AI provider at runtime because the provider can only be changed through configuration. The system also does not perform structural or quality validation of the generated chat answer, detect duplicate answers, or apply content-safety and moderation filtering to the AI output. In addition, there is no response caching mechanism, so the same question may be sent to the AI service again each time instead of reusing a previous response. The chat also does not use streaming output, meaning the student must wait until the complete response is generated before seeing the answer. Finally, when the system uses a fallback Mock answer, there is no visible notice informing the student that the displayed response is a demo or fallback answer.

7. Current Development Progress

The status below is based only on code that exists in the submitted archive. Nothing is marked Completed unless the code supports it.

AI Chat Component	Status	Description
AI Platform Integration	Completed	Hugging Face is fully wired in aiService.js with the router URL, the Bearer key, the request builder, and the provider map. Four alternative platforms and an offline Mock provider are also implemented.
AI Model Integration	Completed	meta-llama/Llama-3.1-8B-Instruct is configured with an HF_MODEL environment override and a hard-coded default, and is sent correctly in the request body with temperature 0.4.
AI Chat Interface	Completed	Tutor.jsx provides an input box, Enter-key submit, an Ask button disabled while loading, four example question buttons, a loading indicator, a formatted answer card, and text-to-speech.
Backend AI Integration	Completed	POST /api/ai/tutor is registered with optionalProtect, validates the input, loads memory, calls askAI(), saves history, awards XP, and returns clean JSON.
Prompt Engineering	Completed	A shared BASE_RULES block plus a chat-specific TASK block define role, tone, language behaviour, output structure, and personalisation rules. The user prompt is built dynamically with memory injection.

AI Chat Component	Status	Description
AI Response Handling	Partially Implemented	Success, empty responses, API errors, network failures, and timeouts are all handled with a graceful Mock fallback. Rate limits are detected but not retried. Failures are hidden from the user behind an HTTP 200 response.
AI Output Validation	Partially Implemented	Only an empty / whitespace check (isUsable) exists. There is no structure check, no duplicate detection, no length or quality threshold, no moderation, and no sanitisation.
Chat Input Validation	Partially Implemented	Empty input is blocked on both the frontend (message.trim()) and the backend (HTTP 400 "message is required"). There is no length limit beyond the 1 MB express.json limit, and no topic filter.
Chat History Storage	Completed	Every chat message is saved to the ChatHistory collection with the feature, question, answer, and topic, and is retrievable through GET /api/ai/history for logged-in users.
Chat Personalisation (memory)	Partially Implemented	Memory notes are written and read correctly, but nothing in the archive ever writes to the WeakTopic collection, so half of the personalisation context stays permanently empty.
Multi-turn Conversation	Not Implemented	No message list, no conversation state, and no previous messages sent to the model. Each question is answered

AI Chat Component	Status	Description
		independently, so follow-up questions are not supported.
Streaming Responses	Not Implemented	The frontend waits for the complete answer. Not found in the current project implementation.
Response Caching	Not Implemented	Listed in the Readme as a future feature (Redis cache), but no caching code exists.
Automated Testing of the Chat	Not Implemented	No test files and no test script were found in the archive.
AI Environment Documentation	Not Implemented	No backend .env or .env.example file is present, so HF_API_KEY and HF_MODEL are not documented anywhere in the project.

Overall Assessment

The core AI Chat pipeline is complete and working from end to end: a student can type a question, the backend routes it through a clean abstraction layer to Hugging Face, and a structured beginner-friendly answer is displayed and can be read aloud. The prompt design is genuinely thoughtful, and the fallback design means the feature can be demonstrated even without an API key or internet connection. The main gaps are in validation and honesty of failure reporting rather than in the integration itself. The three items most worth fixing before final submission are: adding a flag to the API response so the student knows when a demo answer is shown, writing to the WeakTopic collection so the personalisation is complete, and updating the Readme so its listed AI providers match the code.

8. GitHub Repository Link

https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer